Learning Taylor Series Expansions via Sequence-to-Sequence Models

An LSTM and Transformer Comparative Study

Darisi Surya Teja

Mathematics and Computing (MC23BT012)
Indian Institute of Technology, Dharwad

April 20, 2026

Outline

1. Introduction and Motivation
2. Dataset
3. Models
4. Results
5. Timing Benchmark
6. Discussion and Conclusion

The Taylor Series

Given a smooth function f and an expansion point a:

$$f(x) \approx \sum_{k=0}^{n} \frac{f^{(k)}(a)}{k!}(x-a)^k$$

Classical computation (SymPy)

- Exact symbolic differentiation
- Serial, one function at a time
- Cost grows with expression depth
- Requires closed-form input

This work: Neural Prediction

- Learns the mapping from data
- Fixed inference time on GPU
- Batch-parallel processing
- Generalises beyond symbolic class

Central question

When we already have a formula, Why predict?

Motivation (1/2): Scale and Complexity

Reason 1: Throughput at Scale

SymPy processes functions one by one. The Transformer processes a batch of N functions in the same time as one.

Method	1 fn	16 fns
Formula-Based	∼35ms	1,270ms
Seq-to-Seq Model	∼35ms	543ms

Our measured results: 2.3× batch speedup.

Reason 2: Constant Inference Time

SymPy cost scales with expression depth. Transformer cost is fixed regardless of nesting.

Category	SymPy	TF
Simple	8.6ms	29.9ms
Moderate	34.6ms	28.4ms
Very complex	102ms	49ms

2.1× average speedup on very complex functions.

Motivation (2/2): Generality and Learning

Reason 3: Beyond Closed Form

SymPy needs an explicit symbolic expression.

SymPy cannot handle:

- Functions defined by ODEs/PDEs
- Black-box simulators
- Implicit function definitions

A neural model can be extended to these settings using sampled evaluations as input rather than symbolic formulae.

Reason 4: Structural Learning Transfers

Correctly predicting Taylor coefficients requires learning rules of calculus:

- $\sin(x)$ has no even Maclaurin terms
- e^x has all positive coefficients
- Composite derivatives involve convolutions

This structural understanding transfers to harder tasks such as recovering analytical expressions from observed data distributions.

Dataset: Generation and Statistics

12,000 samples generated using SymPy

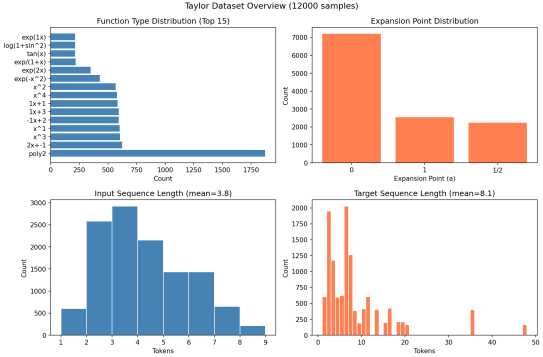
Function families:

- Polynomials: x^n, $ax^2 + bx + c$ (largest category)
- Trigonometric: $\sin(kx)$, $\cos(kx)$, $\tan(x)$
- Exponential: e^x, e^{2x}, e^{x^2}, e^{-x^2}
- Logarithmic: $\ln(1 + x)$, $\ln(1 + x^2)$
- Composites: $e^{\sin x}$, $\cos(\sin x)$, $\sin(x^2)$

Expansion points: $a \in \{0,\ 1/2,\ 1\}$

Split:

- Train: 8,800 Val: 2,200 Test: 1,000



Dataset: Prefix Notation and Tokenization

Both input and output serialised in prefix (Polish) notation

Infix form	Prefix tokens	
$\sin(x)$	`sin x`	Input
$x - \frac{1}{6}x^3$	`+ x * -1/6 pow3 x`	$\sin(x)$ Taylor
$e^x + \sin(x)$	`+ exp x sin x`	Input
$1 + 2x + \frac{x^2}{2}$	`+ 1 + * 2 x * 1/2 pow2 x`	$e^x + \sin x$ Taylor

Advantages:

- No brackets needed
- Unambiguous and recursively structured
- Natural for left-to-right generation

Tokenizer: 118 tokens

- Special: `<PAD>` `<BOS>` `<EOS>`
- Unary/binary operators
- Integers -10 to 10
- Rationals: $1/6$, $-1/6$, $1/24$, $\ldots$

Model Architectures

LSTM Encoder-Decoder

Encoder	BiLSTM, 2 layers
Decoder	LSTM, 2 layers
Hidden	256
Embed	128
Params	**3,597,174**
Dropout	0.3

Bidirectional encoder compresses input to fixed-size context. Decoder generates tokens conditioned on hidden state. Shared embedding matrix.

Transformer Encoder-Decoder

Enc/Dec	3 layers each
d_{model}	128
Heads	4
FFN dim	256
Params	**692,854**
Dropout	0.1

Pre-norm layers.
Sinusoidal positional encoding.
Shared embedding matrix.
5.2× fewer parameters than LSTM.

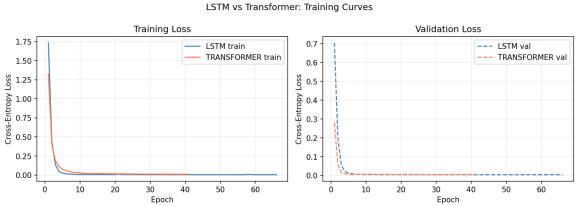
Training Configuration

- **Optimizer:** AdamW
- **Learning Rate:** 3×10^{-4}
- **Scheduler:** Cosine Annealing
- **Gradient Clipping:** 1.0
- **Early Stopping:** Patience $= 10$
- **Hardware/GPU:** RTX 3050 4GB

Note

Both models are trained using these identical settings.

Training Dynamics



LSTM: 66 epochs to converge

- Epoch 1 val loss: 0.7035
- Best val loss: **0.0045**
- Steady, monotone descent

Transformer: 41 epochs to converge

- Epoch 1 val loss: 0.2794
- Best val loss: **0.0046**
- Faster early convergence

Transformer converges 38% faster with nearly identical final loss and 5.2× fewer parameters.

Evaluation Results (1,000 Test Samples)

Table: Evaluation on Test set.

Model	Token Acc. (%)	Seq. Acc. (%)	BLEU-4
LSTM (3.6M params)	78.26	96.00	75.72
Transformer (693K params)	**79.57**	**97.20**	**84.25**

Transformer wins on all three metrics

- +1.31% token accuracy
- +1.20% sequence accuracy
- +8.53 BLEU-4 points

Why the Transformer wins

Cross-attention allows every decoder position to attend directly to every encoder position. For $e^{\sin(x)}$, the x^4 coefficient token attends simultaneously to `exp` and `sin` in the input, implies no information bottleneck.

Prediction Examples

Table: Transformer predictions on selected test inputs.

Input	Predicted	Reference	Correct?
x^4	x^4	same	✓
$\sin(2x)$	$-\frac{4}{3}x^3 + 2x$	same	✓
$e^x + \sin(x)$	$\frac{x^4}{24} + \frac{x^2}{2} + 2x + 1$	same	✓
e^x	$\frac{x^4}{24} + \frac{x^3}{6} + \frac{x^2}{2} + x + 1$	same	✓
e^{2x} at $a = 1$	$\frac{2e^2}{3}x^4 - \frac{4e^2}{3}x^3 + \cdots$	$\frac{2}{3}x^4 + \frac{4}{3}x^3 + \cdots$	×

Notable failure: e^{2x} at $a = 1$

Both LSTM and Transformer predict an expansion containing e^2 factors. But the correct answer is a clean polynomial.

Timing Setup and Per-Function Results

Experimental Setup

- 16 functions across 4 complexity levels
- Accuracy verified (200 test samples)

Category-wise Averages

Category	SymPy	TF	Verdict
Simple	8.6 ms	29.9 ms	SymPy
Moderate	34.6 ms	28.4 ms	TF
Complex	32.8 ms	57.0 ms	Mixed
Very Complex	102 ms	49 ms	TF ($2.1\times$)

Per-Function Highlights

Function	SymPy	TF	Verdict
$x^3 + x$	2.4 ms	12.7 ms	SymPy
$\exp(\sin x)$	54.5 ms	40.7 ms	TF
$\exp(\sin(x^2 + \cos x))$	79.8 ms	23.2 ms	TF ($3.4\times$)
$(\sin x + \cos x)^3 e^{-x^2}$	174 ms	31.2 ms	TF ($5.6\times$)

Timing: Batch Inference and Accuracy

Batch inference (16 functions)

Method	Time	
SymPy (sequential)	1,270ms	baseline
Transformer (batch)	543ms	2.3× faster

Accuracy verified during timing

On 200 fresh test samples:

- Token accuracy: **86.95%**
- Sequence accuracy: **96.50%**
- Exact matches: **193/200**

Speed advantage does not compromise prediction quality.

Traditional v/s Transformer

Simple Functions: Sympy wins (<10ms). GPU overhead dominates for Transformer.

Moderate complexity: Comparable. Crossover around 30-40ms.

Complex Functions: Transformer mostly wins (SymPy >75ms). Speedups up to 5.6× on individual functions.Transformer gets advantage due to GPU parallelism.

Limitations and Future Work

Known limitations:

- Training data limited to a bounded function family. Both models fail on e^{2x} expanded at $a = 1$.
- Imbalanced expansion point distribution (60% $a = 0$, 20% each for $a = 1/2$ and $a = 1$) hurts non-Maclaurin performance.

Future directions:

- Balanced training distribution across expansion points
- Multivariable functions
- Extension to functions defined by differential equations (no symbolic form required)

Summary

What was built

Complete seq2seq pipeline: SymPy dataset generation, 118-token prefix tokenizer with rational coefficients, BiLSTM and Transformer models, and timing benchmark against SymPy.

Key results

Metric	LSTM	TF
Token Acc (%)	78.26	**79.57**
Sequence Acc (%)	96.00	**97.20**
BLEU-4	75.72	**84.25**
Parameters	3.6M	**693K**
Epochs	66	**41**

Timing takeaway

- SymPy time grows with complexity
- Transformer time is roughly constant
- Very complex: 2.1× faster on average
- Best case: 5.6× on $(\sin x + \cos x)^3 e^{-x^2}$
- Batch (16 fns): 2.3× faster
- Accuracy maintained: 193/200 exact at speed

Transformer wins on accuracy, parameter efficiency, convergence speed, and inference throughput.

Thank You!

Open for Discussions

Contact:
Surya Teja Darisi
mc23bt012@iitdh.ac.in